

Infrastructure & Deployment

▼ Relevant source files

Backend/STUDER/Dockerfile

Backend/STUDER/pom.xml

Docker/docker-compose.yml

Docker/nginx.conf

The STUDER platform utilizes a containerized architecture managed by Docker Compose to ensure consistency across development and production environments. The infrastructure is designed with network segmentation to isolate internal services from public-facing components, while Nginx serves as the unified entry point for routing traffic between the Angular frontend and the Spring Boot backend.

System Infrastructure Overview

The deployment architecture consists of six primary containers communicating over two internal bridge networks. The `nginx` service acts as the reverse proxy, exposed on port `1583`, routing requests to either the `frontend` or the `app-be` services based on the URL path.

Deployment Topology

The following diagram illustrates the relationship between Docker services and the internal network isolation.

Service Connectivity Diagram

Sources: `Docker/docker-compose.yml` 2-86 `Docker/nginx.conf` 28-51

Container Services & Orchestration

The `docker-compose.yml` file defines the orchestration of the entire stack. It manages environment variable injection for sensitive credentials (database passwords, JWT secrets) and ensures service dependency ordering.

Service	Image / Build Source	Role
db	postgres:16	Relational storage for user data, discussions, and notifications.
app-be	../Backend/STUDER/	Primary Java backend running with the <code>prod</code> profile.
studer-media	../Backend/STUDER-MEDIA/	Python microservice for image processing.
frontend	../Frontend/STUDER/	Nginx-hosted Angular SPA.
minio	minio/minio	S3-compatible storage for media assets.
nginx	nginx:alpine	Global reverse proxy and SSL termination point.

For details on specific service configurations and multi-stage builds, see [Docker Compose & Container Configuration](#).

Sources: `Docker/docker-compose.yml` 3-86 `Backend/STUDER/Dockerfile` 1-14

Network & Routing Logic

Traffic routing is handled by Nginx using `upstream` blocks. The configuration defines two primary listeners: one on port `1583` for standard access and another on port `5712` for backend-specific routing.

- API Routing:** Requests starting with `/api/` are forwarded to the `app_upstream` (the Spring Boot application on port `8081`).
- Frontend Routing:** All other requests are directed to the `frontend_upstream` (the Angular application).
- WebSockets:** The configuration includes `Upgrade` and `Connection` headers to support WebSocket handshakes required by modern web frameworks.

For details on the proxy configuration and header management, see [Nginx Reverse Proxy](#).

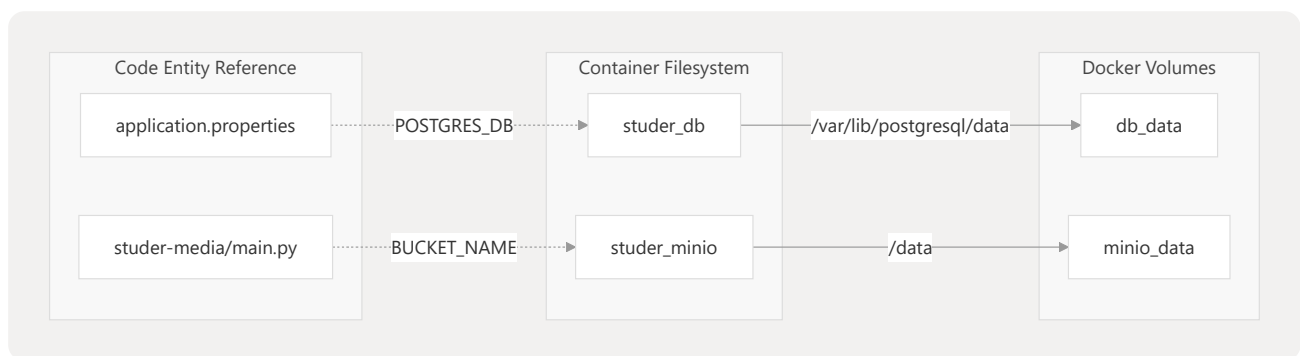
Sources: `Docker/nginx.conf` 18-26 `Docker/nginx.conf` 33-50

Persistence & Storage

STUDER implements volume persistence to ensure data survives container restarts. Two named volumes are defined in the root configuration:

1. `db_data` : Maps to `/var/lib/postgresql/data` to persist the PostgreSQL state.
2. `minio_data` : Maps to `/data` within the MinIO container to persist uploaded images and avatars.

Data Persistence Mapping



Sources: `Docker/docker-compose.yml` 87-89 `Docker/docker-compose.yml` 13-14

`Docker/docker-compose.yml` 79-80

Build Pipeline

The backend utilizes a multi-stage `Dockerfile` to minimize the production image size. It first uses a Maven image to compile the source and then copies the resulting `.jar` into a lightweight JRE image.

1. **Stage 1 (Build):** Uses `maven:3.9.6-eclipse-temurin-17-alpine` to run `mvn clean package`.
2. **Stage 2 (Run):** Uses `eclipse-temurin:17-jre-alpine` to execute the `STUDER-0.0.1-SNAPSHOT.jar`.

For details on the frontend build toolchain (esbuild/vite) and testing setup, see [Frontend Build Configuration](#).

Sources: Backend/STUDER/Dockerfile 1-13 Backend/STUDER/pom.xml 12-13